1 Application Implementation

The preceding sections established that running a full Large Language Model on consumer mobile hardware is impractical, and that a client–server architecture — the device acting as a user-facing interface to a remotely hosted model — is the most consistent way to deliver an LLM-powered experience on mobile. This section documents the realisation of that architecture in a working application.

MotivBot administers the well-being battery described above and returns individualised supportive feedback generated by a remote language model. The participant first describes their emotional state in free text, then completes the 62-item questionnaire, after which both the narrative and the structured responses are transmitted to the model. Turkish and Persian are supported, the latter requiring right-to-left layout. The application was built twice: natively for iOS in Swift, and subsequently for Android in Kotlin.

1.1 The Two Native Languages

Apple relied on Objective-C until Swift was introduced at the 2014 Worldwide Developers Conference. Swift was designed to eliminate the characteristics of Objective-C that most often produced defects, and three are directly relevant here [1]. **Optionality** requires that any variable which may hold no value be declared with the `?` suffix, and the compiler refuses to read it unwrapped, eliminating a class of null-dereference crashes before the program runs. **Value semantics** mean a `struct` is copied on assignment rather than shared, so passing a response to another component cannot let that component mutate the original. **Structured concurrency** via `async/await` allows a network call to be written sequentially, replacing the nested completion handlers that previously fragmented control flow. The interface uses SwiftUI, Apple's declarative framework, in which the developer describes what the interface should contain for a given state rather than issuing instructions to mutate it. Development requires Xcode on macOS; the application targets iOS 26.2.

Android development was historically conducted in Java. Kotlin, developed by JetBrains, became an official Android language in 2017 and Google's preferred language in 2019 [2]. Its design goals closely parallel Swift's, which is what made the port tractable: nullability is expressed in the type system in the same way, the `data class` serves the role of the Swift `struct`, and coroutines provide concurrency, with `suspend` functions corresponding to Swift's `async` functions. The interface uses Jetpack Compose [3], close enough to SwiftUI in philosophy that most screens translated structurally rather than being redesigned. The application targets API 35, supports API 26 and above, and compiles against JVM 17.

Concern	**iOS (Swift)**	**Android (Kotlin)**
Null safety	Optionals (`?`, `guard let`)	Nullable types (`?`, `?.`)
Data modelling	`struct`, `Codable`	`data class`, `@Serializable`
Concurrency	`async/await`, `Task`	`suspend`, coroutines, `Flow`
UI framework	SwiftUI (declarative)	Jetpack Compose (declarative)
State observation	`@Observable`	`StateFlow`
Thread guarantee	`@MainActor`	`Dispatchers.Main`
Toolchain	Xcode (macOS only)	Android Studio, Gradle

Table 1: Correspondence between the two native language ecosystems.

1.2 Cross-Platform Alternatives

The survey above examined Flutter, React Native and Kotlin Multiplatform, observing that cross-platform frameworks reduce cost at the price of indirect hardware access. That analysis preceded implementation; what building the application revealed refines it.

The iOS implementation comprises 13 Swift files totalling 2,059 lines; the Android implementation,

35 Kotlin files totalling 4,020 lines, an increase that overstates duplicated effort since much of it reflects explicit dependency injection and a separated data layer rather than restated logic. The more serious cost was not the writing but the **divergence**: several defects were corrected in the Kotlin implementation — most consequentially, that the model was being instructed in English regardless of interface language — and none propagated to the Swift implementation automatically. Two codebases holding the same intent separate under maintenance unless deliberate effort keeps them aligned, and that effort is a permanent tax rather than a one-time cost.

Kotlin Multiplatform occupies a position between the two extremes surveyed: rather than sharing the entire application, it shares only the layers containing no platform-specific concerns — here the questionnaire definitions, prompt construction and network client — while the interface remains fully native [4]. This suits the application, because the layer that most needs to remain consistent is precisely the one containing no user interface: the instrument definitions and their scoring must be identical across platforms or the research data is not comparable. Native development was nonetheless retained, because the iOS application existed before the Android requirement arose, because right-to-left Persian layout is more reliably obtained from the platform than from a framework's reimplementation of it, and because building the same application twice yields a direct comparison of the two ecosystems.

1.3 LLM Integration

No model executes on the device. The application issues an HTTPS request to a remotely hosted model and renders the response, so the memory-hierarchy constraints examined earlier — the windowing and bundling techniques required to run inference under a constrained DRAM budget — do not arise. The request uses `gpt-4o-mini` with temperature 0.8, a 500-token limit and a 60-second timeout. The temperature is deliberately high: a low value produces near-identical phrasing across sessions, undermining the impression of individualised feedback, while the supportive register is constrained by the system prompt instead. Tokens are not streamed, since the response is short and presented as a single passage.

Each request carries a system message establishing the assistant's role and a user message containing the participant's narrative followed by every item paired with its response and score. The most instructive defect in this project concerned language: the original implementation sent a fixed English system prompt while questionnaire content reached the model in the participant's language, leaving the output language undefined in practice, so a participant reading a Turkish interface could receive an English reply. The correction was to construct the system prompt as a function of the selected language.

```
private fun systemPrompt(language: AppLanguage): String {
    val script = if (language.isRtl)
        " Write in the Persian script, right-to-left, using Persian (not Arabic) digits."
           else ""
    return """
        You MUST write your entire response in ${language.promptName}.$script
        You are a compassionate psychological well-being assistant. ...
        The questionnaire below may reach you in ${language.promptName} or in English;
        either way, your answer must be entirely in ${language.promptName}.
    """.trimIndent()
}
```

Listing 1: Language-conditioned system prompt construction.

The language instruction appears both before and after the assistant persona, because a single instruction at the head of a prompt is followed unreliably once several thousand tokens of questionnaire data intervene. The Persian branch also specifies the digit set, because a model asked to write Persian will otherwise frequently emit Arabic-Indic numerals, which Persian readers find incongruous.

1.4 Technology Stack and Justification

Concern	Library	Reason for selection
iOS		
UI	SwiftUI	Declarative; first-party; no third-party dependency
Networking	URLSession	Built in; native `async`/`await` support
Serialisation	Codable	Compiler-generated; no reflection or manual parsing
Android		
UI	Jetpack Compose [3]	Declarative; closest structural analogue to SwiftUI
Networking	Retrofit + OkHttp [5]	Declarative HTTP interfaces; native `suspend` support
Serialisation	kotlinx.serialization	Compile-time generated, matching `Codable`
DI	Hilt [6]	Compile-time graph validation; lifecycle-aware scoping
Database	Room [7]	Compile-time SQL verification against the schema
Preferences	DataStore	Asynchronous and transactional, unlike SharedPreferences
Testing	JUnit, MockWebServer	Network behaviour testable without a live endpoint

Table 2: Libraries selected for each platform, with justification.

A consistent principle governs these selections: **prefer libraries that move errors from runtime to compile time**. Retrofit validates endpoint declarations during compilation; Room parses SQL and verifies it against the declared schema; Hilt validates the dependency graph before the application runs; kotlinx.serialization generates serialisers at compile time rather than resolving fields reflectively. In each case a failure that would otherwise appear on a participant's device appears instead on the developer's screen — and for an application administering clinical instruments, where a runtime failure mid-assessment loses the session, that is the correct trade-off. Two translations required more than substituting one library for its counterpart: Swift's `URLRequest.timeoutInterval` bounds a request as a whole, whereas OkHttp exposes connect, read and write phases separately, so equivalent behaviour was reconstructed by giving each phase a budget and setting `callTimeout` to cap the total; and the Swift service signalled failure by throwing a `LocalizedError`, whereas the Kotlin data source returns a sealed `AppResult` type, so the compiler enforces that every failure mode is handled at the call site. Correctness is verified by 33 unit tests across four suites, covering questionnaire progression, prompt and CSV construction, the network layer against a mock server, and input validation.

In summary, building natively twice confirmed both the expected benefit and the expected cost: platform behaviour such as right-to-left Persian layout came without effort, while the two codebases began to diverge as soon as defects were corrected in one and not the other, which is why Kotlin Multiplatform is recommended for future work. The most transferable finding, however, concerns LLM integration rather than platform choice: the language defect described above passed every conventional test — the code compiled, the request succeeded, and a well-formed response was returned; only its language was wrong. Applications built on language models therefore require a form of verification that conventional testing does not supply, directed at whether the model was asked the right question rather than at whether the request was correctly sent.

References

[1] Apple Inc., *The Swift Programming Language*. [Online]. Available: https://docs.swift.org/swift-book/

[2] JetBrains, *Kotlin Documentation*. [Online]. Available: https://kotlinlang.org/docs/home.html

[3] Google, *Jetpack Compose*. [Online]. Available: https://developer.android.com/compose

[4] JetBrains, *Kotlin Multiplatform*. [Online]. Available: https://kotlinlang.org/docs/multiplatform.html

[5] J. Wharton et al., "Retrofit: A type-safe HTTP client for Android and Java," Square. [Online]. https://square.github.io/retrofit/

[6] Google, *Dependency Injection with Hilt*. [Online]. Available: https://developer.android.com/training/dependency-injection/hilt-android

[7] Google, *Room Persistence Library*. [Online]. Available: https://developer.android.com/training/data-storage/room